

```
!pip install numpy matplotlib scipy
mambajs 0.21.4
Process pip requirements ...
Requirement numpy already handled by conda/micromamba/mamba.
Requirement matplotlib already handled by conda/micromamba/mamba.
Requirement scipy already handled by conda/micromamba/mamba.

#libraries
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
```

Sinusoidal

```
# define functions
def f_sin(A, w, t):
    """
    Returns the value of  $y = A\sin(wt)$  function where
    A = Amplitude
    w = angular frequency
    t = time
    """
    return A*np.sin(w*t)

def f_cos(A, w, t):
    """
    Returns the value of  $y = A\cos(wt)$  function where
    A = Amplitude
    w = angular frequency
    t = time
    """
    return A*np.cos(w*t)

#plot of  $y = A\sin(w*t)$  and  $y = A\cos(w)$  on a single figure
figure, (ax1, ax2) = plt.subplots(1, 2, figsize=(8, 4))

# sine curve params
A1 = 1
freq = 2
w1 = 2*np.pi*freq
t1 = np.arange(0, 2, 0.01)
y1 = f_sin(A1, w1, t1)
```

```

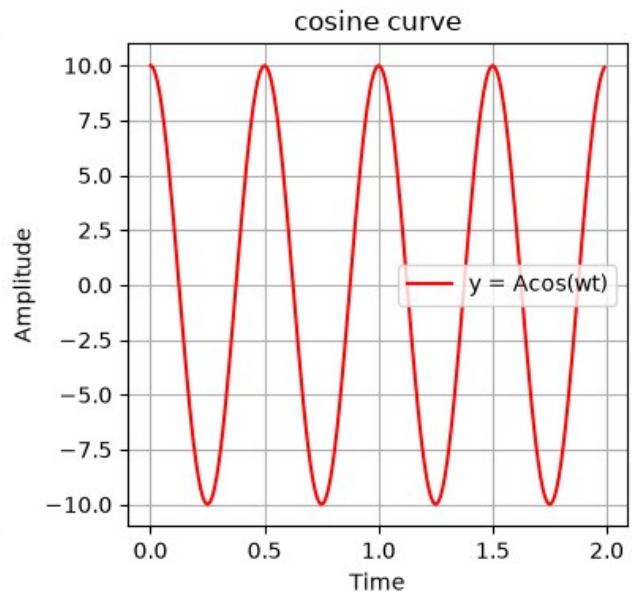
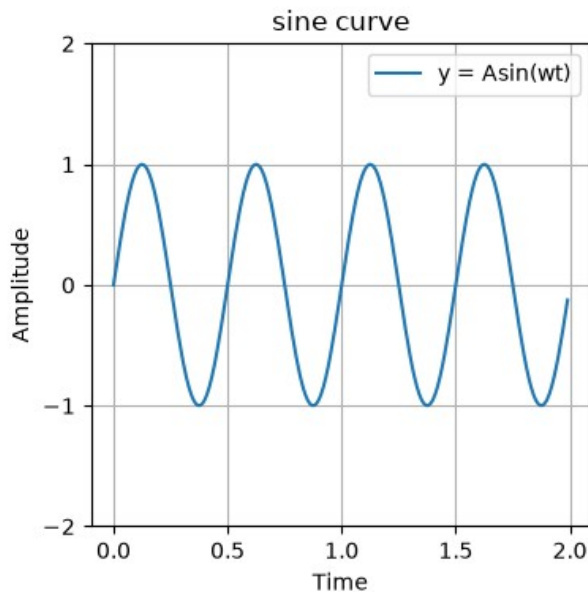
#plotting sine wave
ax1.plot(t1, y1, label="y = Asin(wt)")
ax1.set_yticks(np.arange(-2, 3, 1))
ax1.set_title("sine curve")
ax1.set_xlabel("Time")
ax1.set_ylabel("Amplitude")

ax1.legend()
ax1.grid(True)

#cosine curve params
A2 = 10
freq = 2
w2 = 2*np.pi*freq
t2 = np.arange(0, 2, 0.01)
y2 = f_cos(A2, w2, t2)
ax2.plot(t2, y2, 'r-', label="y = Acos(wt)")
ax2.set_title("cosine curve")
ax2.set_xlabel("Time")
ax2.set_ylabel("Amplitude")
ax2.grid(True)
ax2.legend()

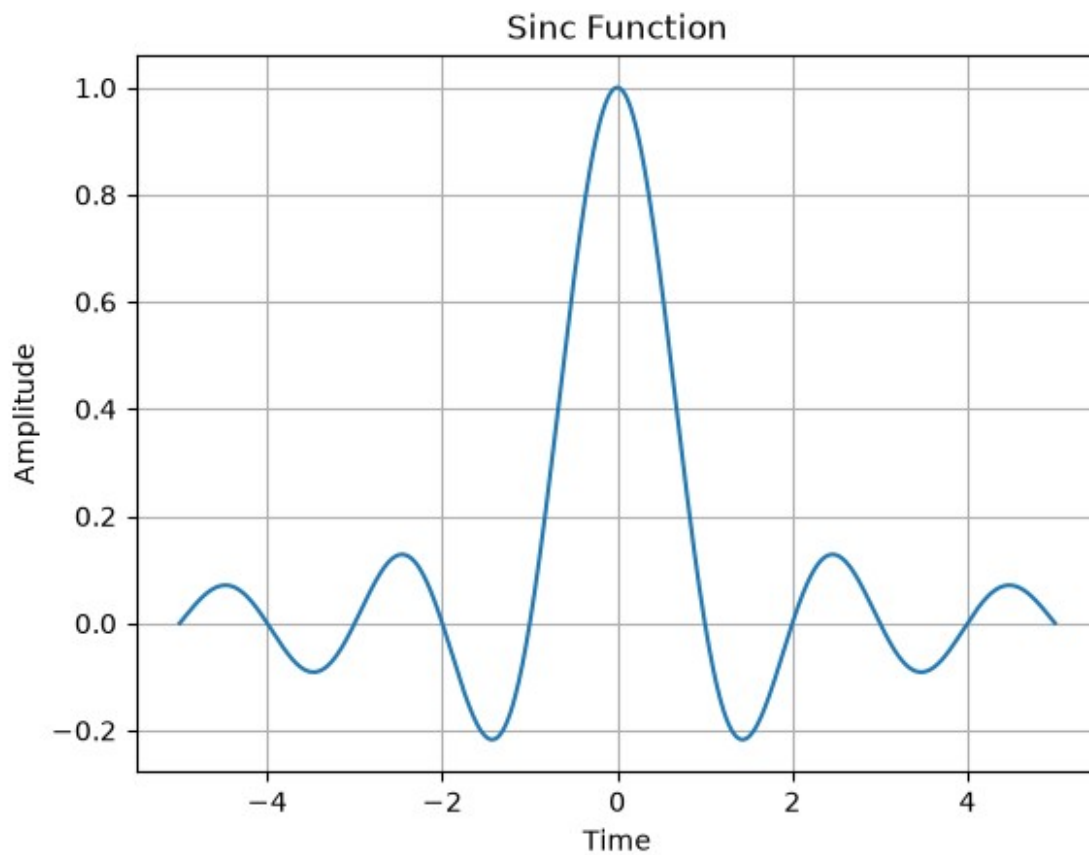
plt.tight_layout()
plt.legend()
plt.show()

```



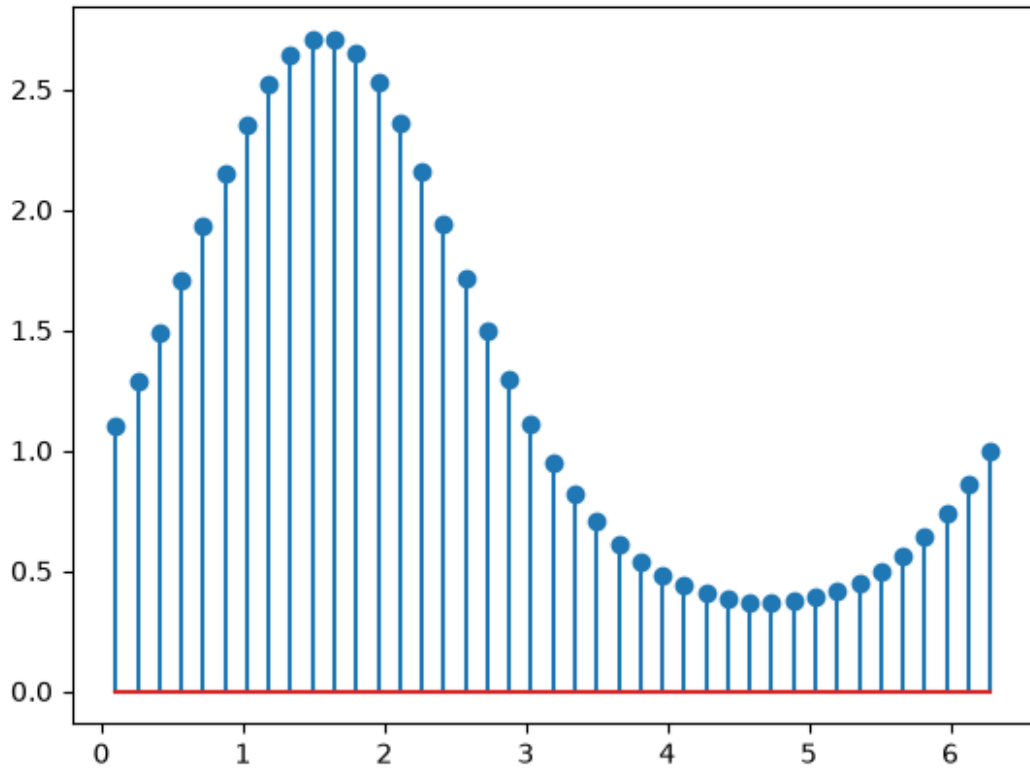
Sinc Function

```
x = np.linspace(-5, 5, 1000)
y = np.sinc(x)
plt.plot(x, y)
plt.title("Sinc Function")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```



Stem Plot

```
x = np.linspace(0.1, 2*np.pi, 41)
y = np.exp(np.sin(x))
plt.stem(x, y)
plt.show()
```



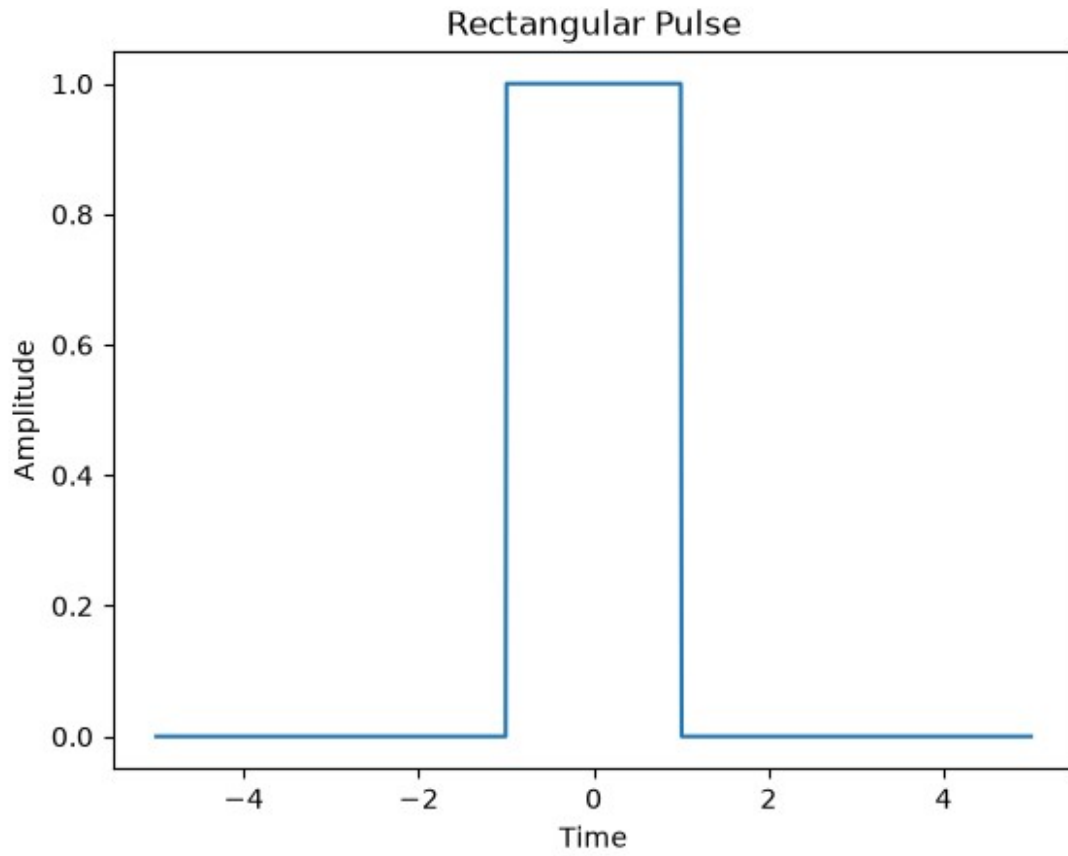
Rectangular Pulse

#define rectangular pulse function

```
def rect_pulse(t, start, end, amp):  
    return np.where((t >= start) & (t <= end), amp, 0)
```

draw a rectangular pulse

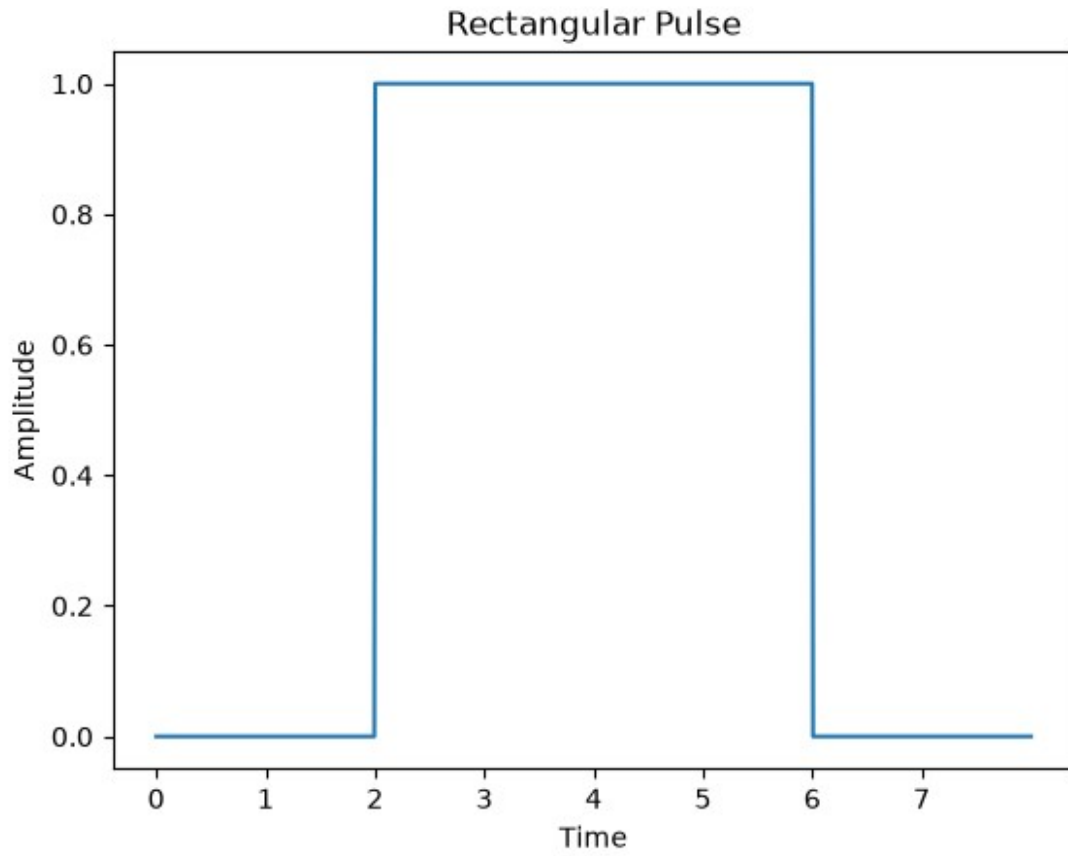
```
start = -1  
end = 1  
amplitude = 1  
t = np.linspace(-5, 5, 1000)  
y = rect_pulse(t, start, end, amplitude)  
plt.plot(t, y)  
plt.title("Rectangular Pulse")  
plt.xlabel("Time")  
plt.ylabel("Amplitude")  
plt.show()
```



Question 01

draw a rectangular pulse starting at $t = 2$ and ends at $t = 6$

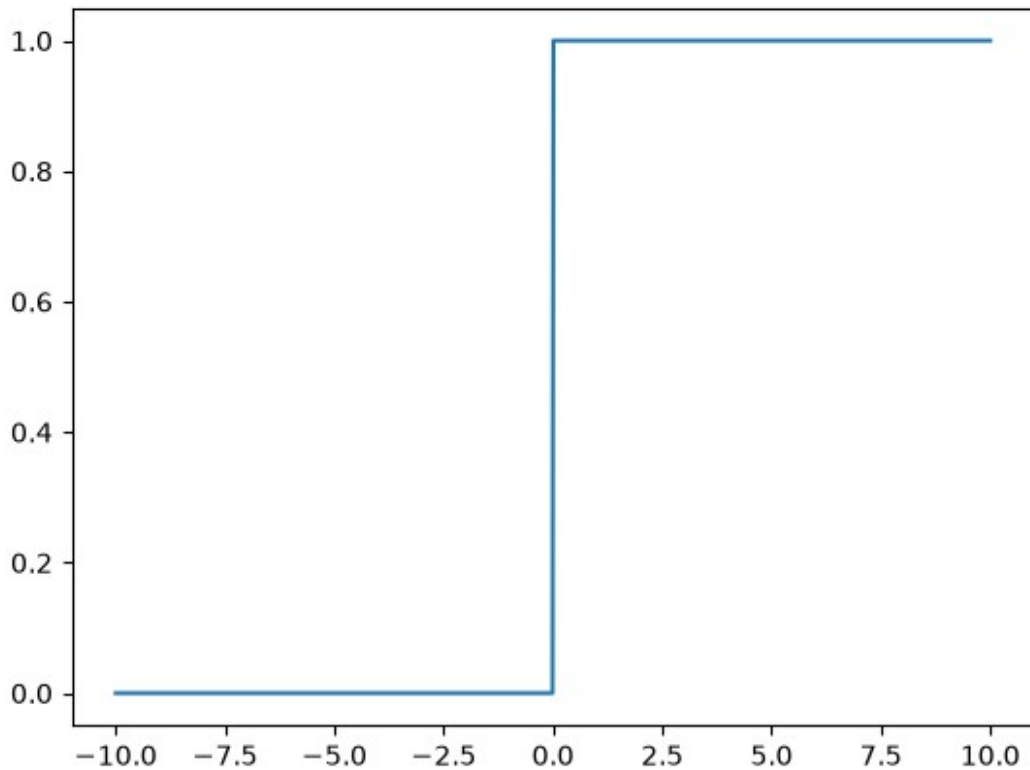
```
start = 2
end = 6
amplitude = 1
t = np.linspace(0, 8, 1000)
y = rect_pulse(t, start, end, amplitude)
plt.plot(t, y)
plt.title("Rectangular Pulse")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.xticks(np.arange(0, 8, 1))
plt.show()
```



Unit Step Function

```
# define unit function
def f_unit(t, start = 0, amp = 1):
    t = t - start
    return amp*(np.where(t >= 0, 1, 0))

t = np.linspace(-10, 10, 1000)
y = f_unit(t)
plt.plot(t, y)
plt.show()
```



Question 02

```
# plot the signal  $x(t) = u(t-2) - u(t-4)$ 

fig, ax = plt.subplots(1, 3, figsize=(9, 3))

t = np.linspace(0, 8, 1000)
y1 = f_unit(t, 2, 1)
y2 = f_unit(t, 4, 1)
y3 = f_unit(t, 2, 1) - f_unit(t, 4, 1)

#params for first plot
ax[0].plot(t, y1, 'b', label="x(t) = u(t-2)")
ax[0].plot(t, y2, 'r--')
ax[0].set_title("x(t) = u(t-2)")

#params for second plot
ax[1].plot(t, y2, 'r', label="x(t) = u(t-4)")
ax[1].plot(t, y1, 'b--')
ax[1].set_title("x(t) = u(t-4)")

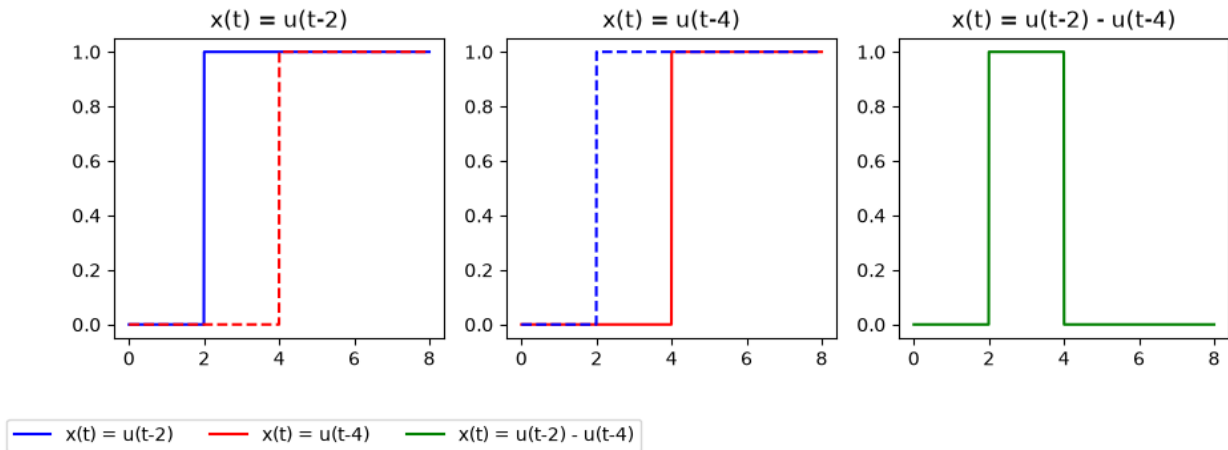
#params for final plot
ax[2].plot(t, y3, 'g', label="x(t) = u(t-2) - u(t-4)")
```

```

ax[2].set_title("x(t) = u(t-2) - u(t-4)")

plt.tight_layout()
fig.legend(
    loc="upper right",
    ncol=3,
    bbox_to_anchor=(0.5, -0.05)
)
plt.show()

```



Ramp Function

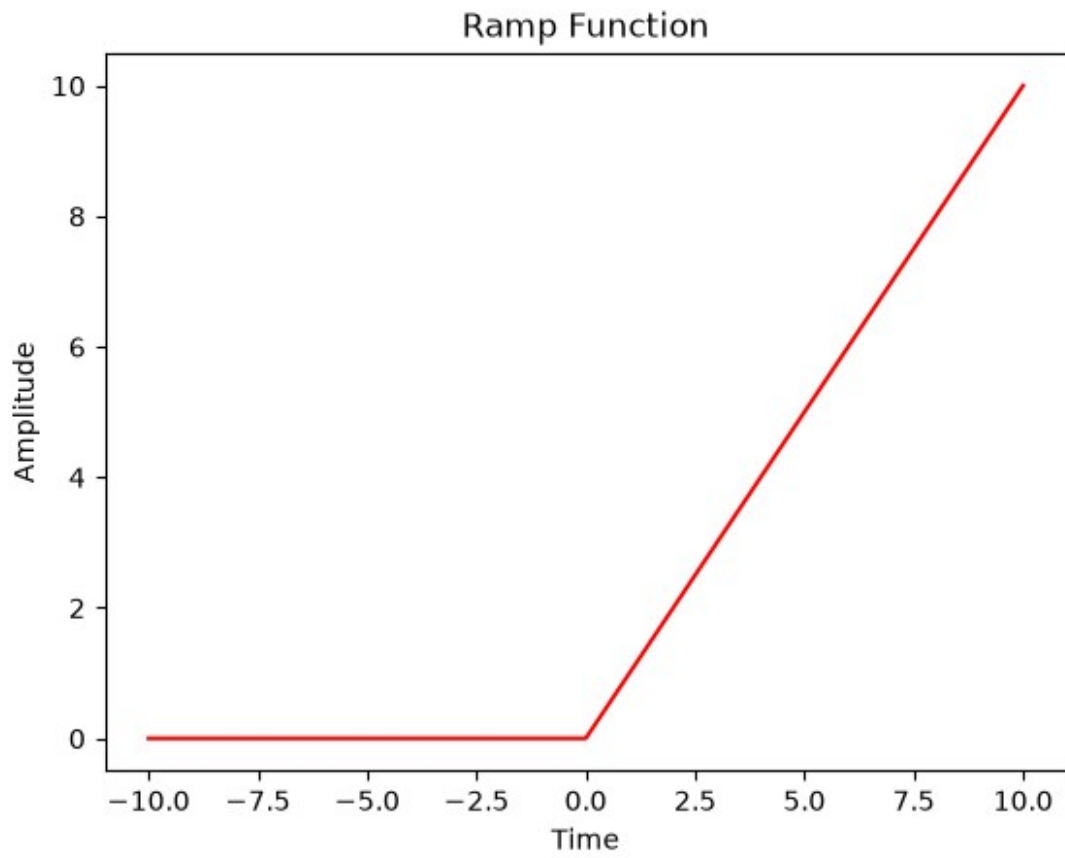
```

# define ramp function

def f_ramp(t, start):
    return np.where(t >= start, t, 0)

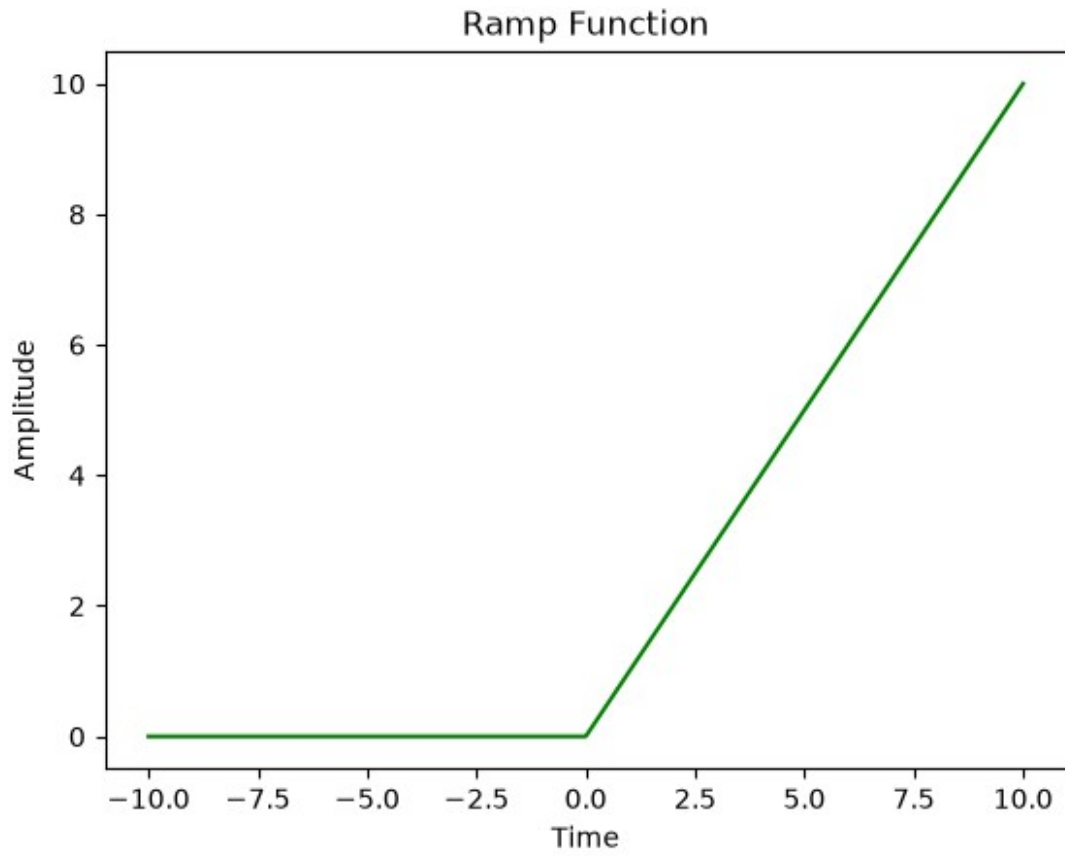
t = np.linspace(-10, 10, 1000)
y = f_ramp(t, 0)
plt.plot(t, y, 'r')
plt.title("Ramp Function")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.show()

```



Draw Ramp Function Using Unit Function

```
t = np.linspace(-10, 10, 1000)
y = t*f_unit(t, 0, 1)
plt.plot(t, y, 'g')
plt.title("Ramp Function")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.show()
```

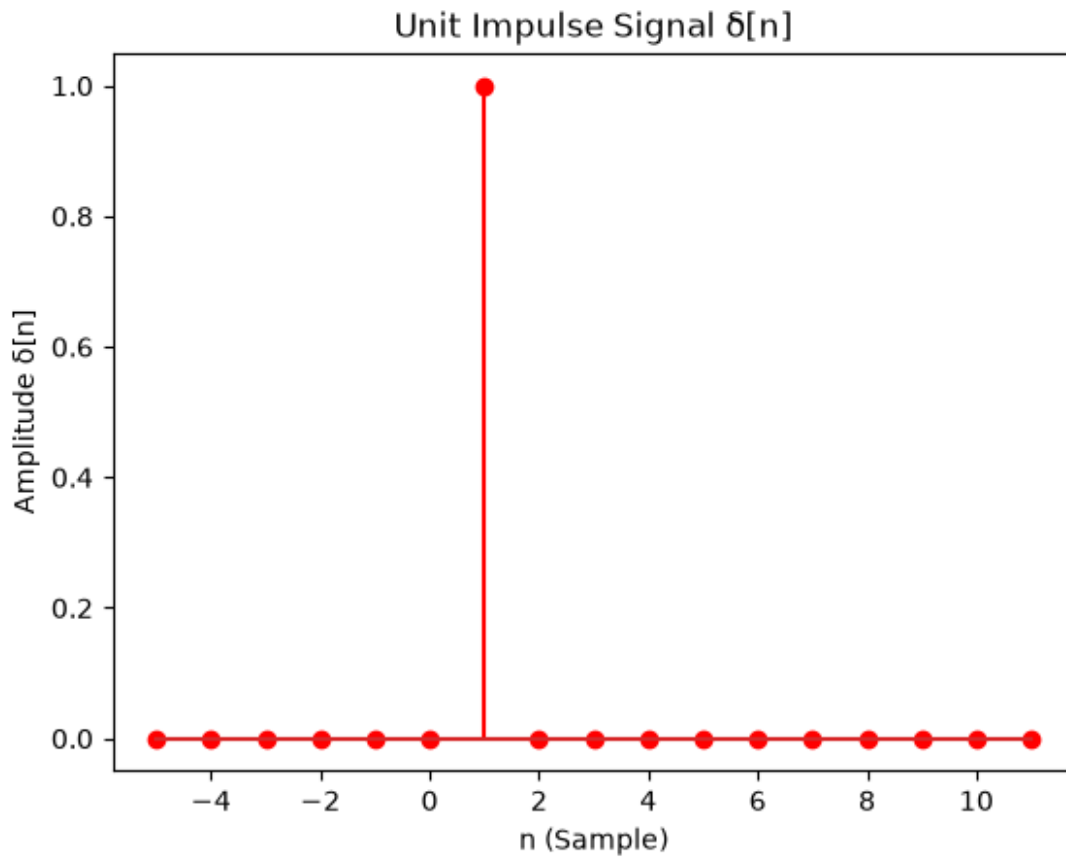


Unit Impulse Signal

```
#define unit impulse Signal

def f_imp(n, pulse):
    return np.where(n == pulse, 1, 0)

n = np.arange(-5, 12, 1)
y = f_imp(n, 1)
plt.stem(n, y, 'r')
plt.xlabel("n (Sample)")
plt.ylabel("Amplitude  $\delta[n]$ ")
plt.title("Unit Impulse Signal  $\delta[n]$ ")
plt.show()
```



Shifting, Scaling, Reversal of Unit Impulse Function

```
# plot the signal  $x(t) = u(t-2) - u(t-4)$ 

fig, ax = plt.subplots(1, 3, figsize=(9, 3))

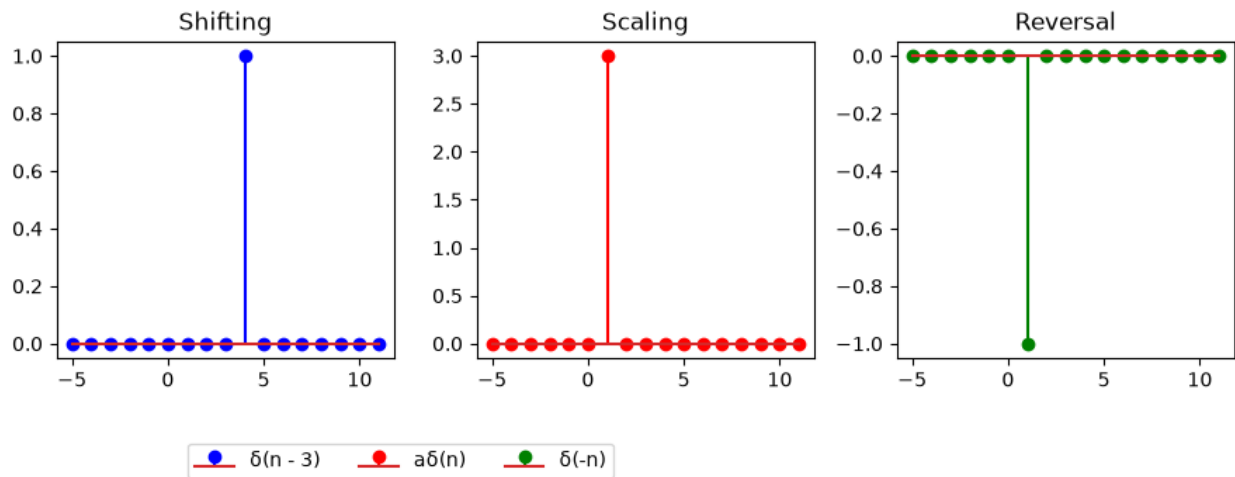
y1 = f_imp(n - 3, 1)
a = 3
y2 = a*f_imp(n, 1)
y3 = - f_imp(n, 1)

#params for first plot (shifting)
ax[0].stem(n, y1, 'b', label=" $\delta(n - 3)$ ")
ax[0].set_title("Shifting")

#params for second plot
ax[1].stem(n, y2, 'r', label=" $a\delta(n)$ ")
ax[1].set_title("Scaling")

#params for final plot
ax[2].stem(n, y3, 'g', label=" $\delta(-n)$ ")
ax[2].set_title("Reversal")
```

```
plt.tight_layout()
fig.legend(
    loc="upper right",
    ncol=3,
    bbox_to_anchor=(0.5, -0.05)
)
plt.show()
```

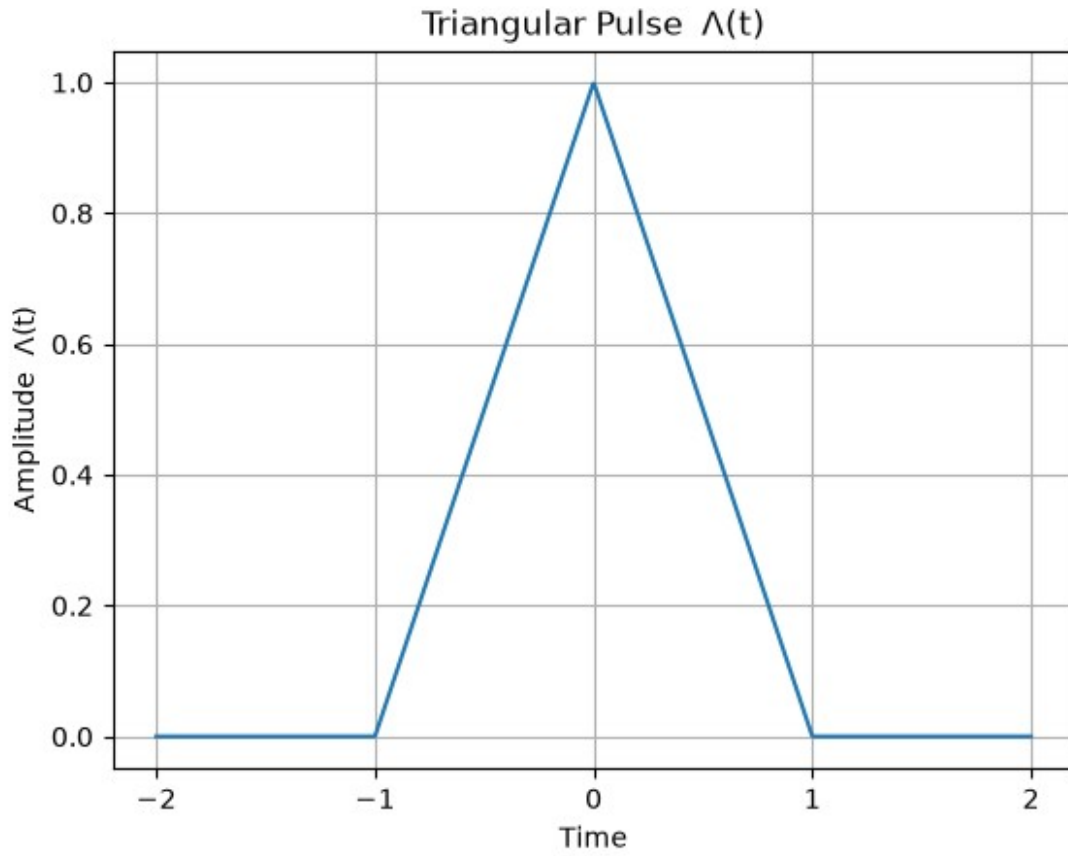


Triangular Pulse

```
#define triangular pulse function

def f_trig(t):
    return np.where(np.abs(t) < 1, 1-np.abs(t), 0)

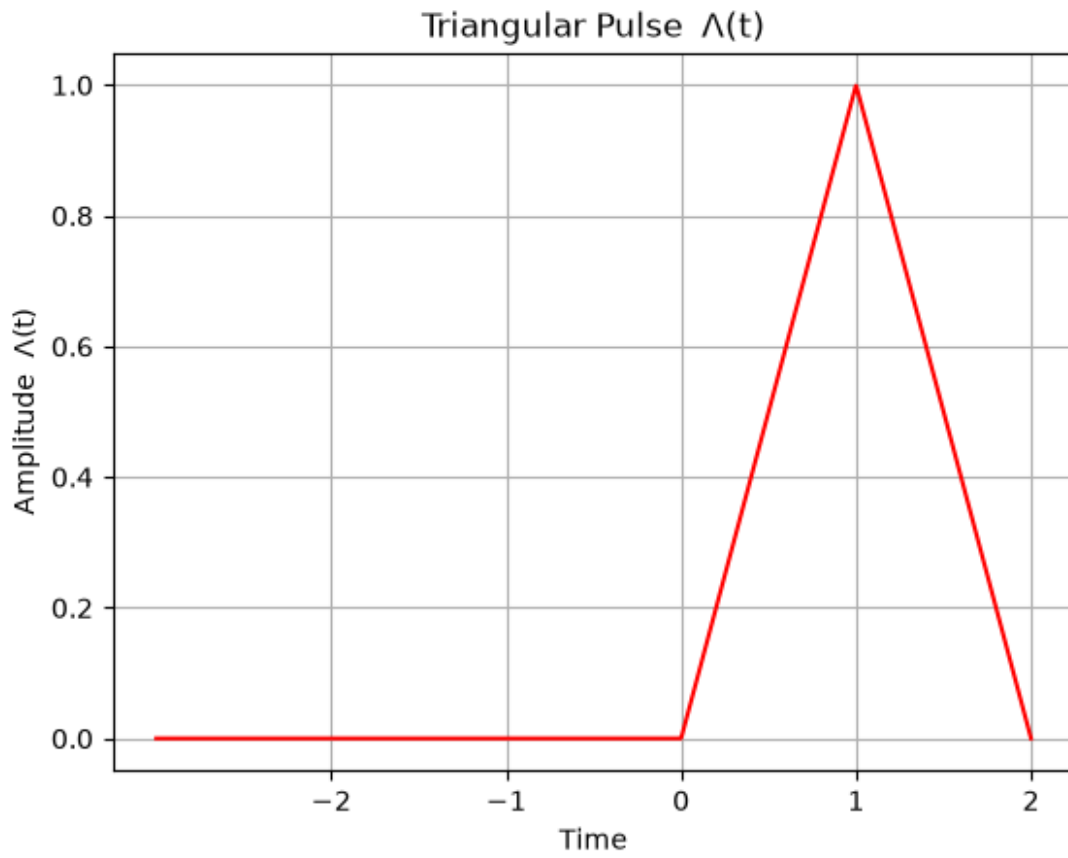
t = np.linspace(-2, 2, 1000)
y = f_trig(t)
plt.plot(t, y)
plt.xticks(np.arange(-2, 3, 1))
plt.grid(True)
plt.title("Triangular Pulse  $\Lambda(t)$ ")
plt.xlabel("Time")
plt.ylabel("Amplitude  $\Lambda(t)$ ")
plt.show()
```



Draw Triangular Pulse Using Ramp and Unit Function

```
t = np.linspace(-3, 2, 1000)

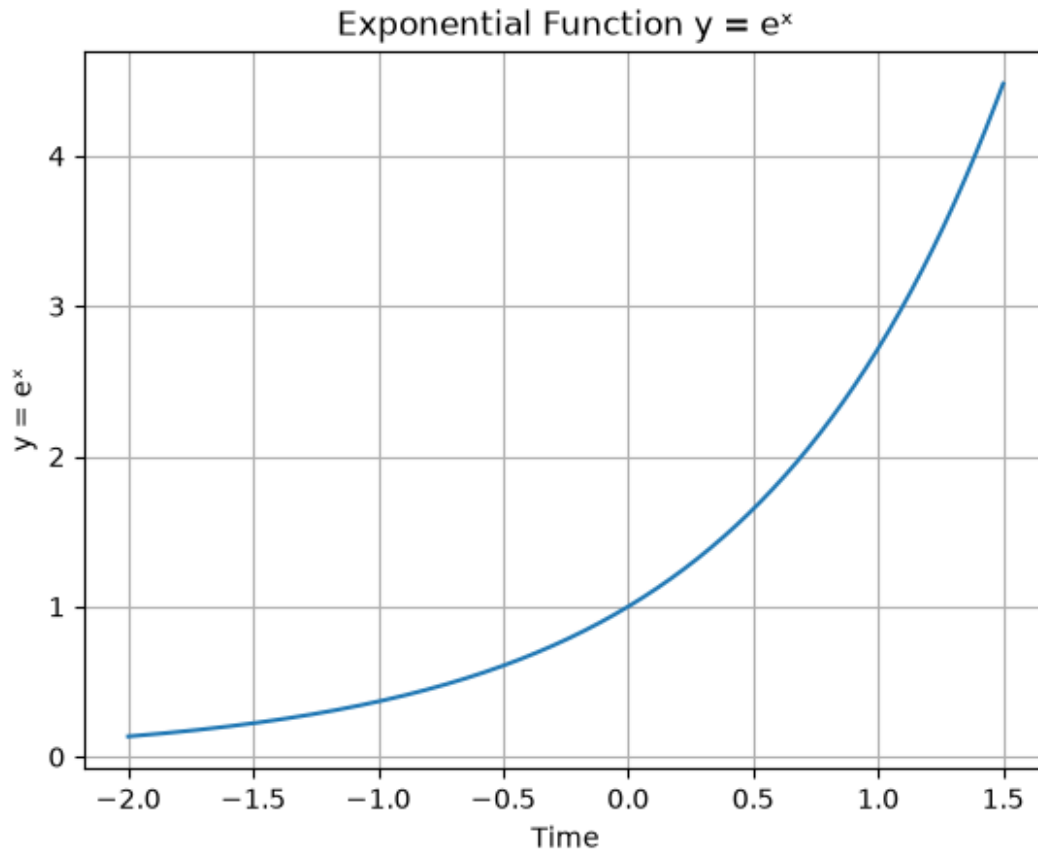
y = (t*f_unit(t) - 2*(t-1)*f_unit(t - 1) + (t-2)*f_unit(t - 2))
#ramp(t) = t*u(t)
plt.plot(t, y, 'r')
plt.xticks(np.arange(-2, 3, 1))
plt.title("Triangular Pulse  $\Lambda(t)$ ")
plt.xlabel("Time")
plt.ylabel("Amplitude  $\Lambda(t)$ ")
plt.grid(True)
plt.show()
```



Exponential Function

```
x = np.linspace(-2, 1.5, 1000)
y = np.exp(x)

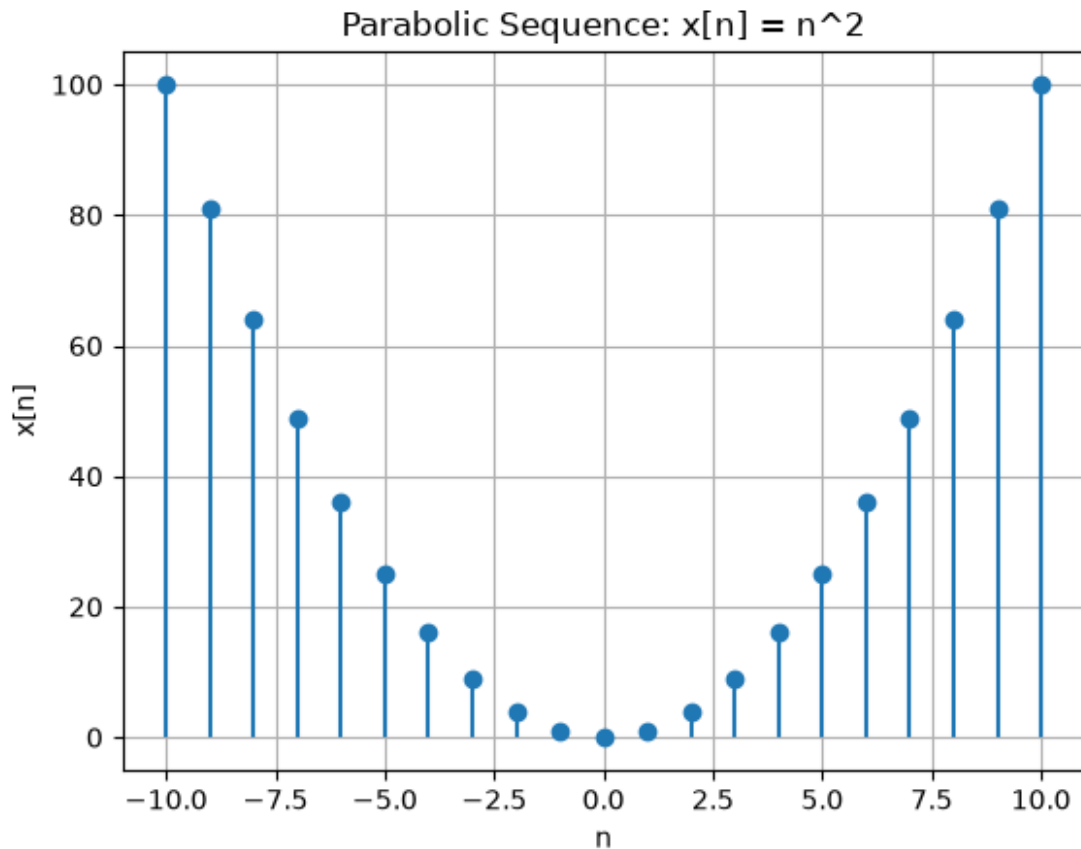
plt.plot(x, y)
plt.title("Exponential Function y = e^x")
plt.xlabel("Time")
plt.ylabel("y = e^x")
plt.grid(True)
plt.show()
plt.show()
```



Parabolic Sequence

```
# Define parabolic sequence function
def parabolic_sequence(n):
    return n ** 2

#plotting parabolic sequence
n = np.arange(-10, 11, 1)
y = parabolic_sequence(n)
plt.stem(n, y, basefmt=" ")
plt.title("Parabolic Sequence: x[n] = n^2")
plt.xlabel("n")
plt.ylabel("x[n]")
plt.grid(True)
plt.show()
```

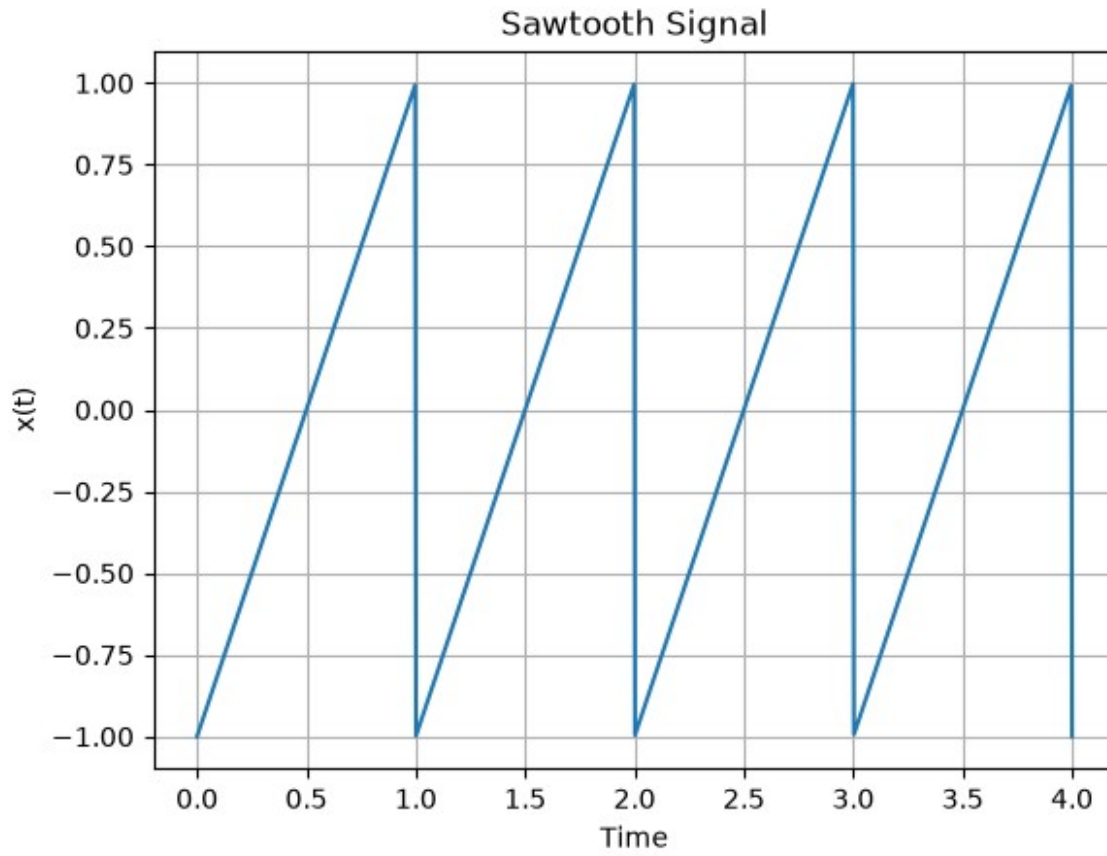


Standard Signals

Sawtooth Signal

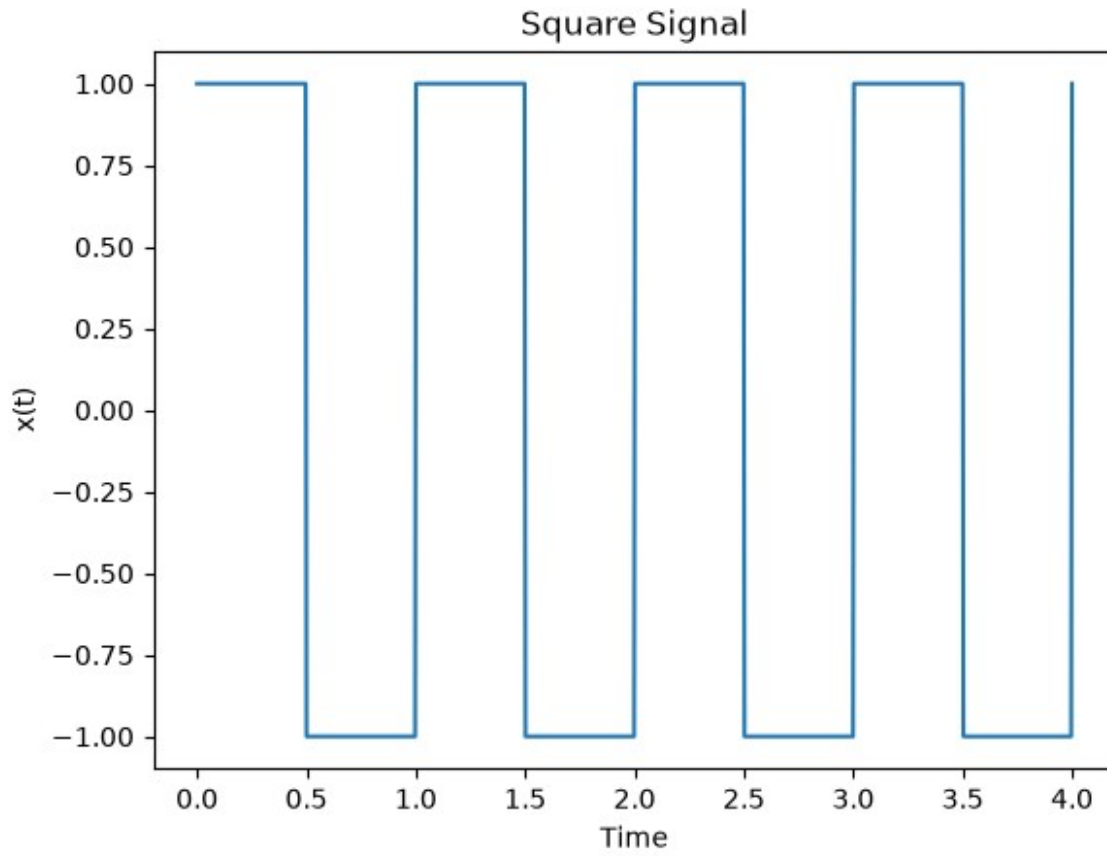
```
t = np.linspace(0, 4, 1000)
freq = 1.0

y = x_sawtooth = signal.sawtooth(2 * np.pi * freq * t)
plt.plot(t, y)
plt.title("Sawtooth Signal")
plt.xlabel("Time")
plt.ylabel("x(t)")
plt.grid(True)
plt.show()
```



Square Wave

```
y = x_sawtooth = signal.square(2 * np.pi * freq * t)
plt.plot(t, y)
plt.title("Square Signal")
plt.xlabel("Time")
plt.ylabel("x(t)")
plt.show()
```



Complex Exponential Function

```
# Parameters
sigma = -0.25
omega = 3.2

x = np.exp((sigma + 1j*omega) * t)

plt.figure(figsize=(8,4))
plt.plot(t, np.real(x), color='red', linewidth=3)

plt.axhline(0, color='black', linewidth=1.5)
plt.axvline(0, color='black', linewidth=1.5)

plt.xlim(-4,4)
plt.ylim(-3,3)

plt.title("Complex Exponential Function")
plt.xlabel("Time")
```

```
plt.ylabel("x(t)")  
  
plt.xticks([])  
plt.yticks([])  
  
plt.box(False)  
plt.tight_layout()  
plt.show()
```

